:3.1

.1

1. **אופרטור < :** (במידה והכיוון של האופרטור מהתפך אז הכוונה היא לגדול מ..)
אנחנו מחייבים שיהיה אופרטור < שכן אחרת לא היינו יכולים לקבוע את יחס הסדר של האיברים, וזה הכרחי כיוון שSortedList הוא מבנה נתונים ששומר על האיברים בסדר ממויין לפי יחס הסדר שלהם, אילולא לא היה לנו אופרטור כזה לא יכלנו לקבוע בעת ההכנסה לאן האיבר החדש יכנס, והמושג רשימה ממויינת לא היה מקבל משמעות בכלל.

הערה:
יש לציין כי יכלנו באותה מידה להשתמש באופרטורי סדר אחרים, אבל במטלה ביקשתם ספציפית גדול מ..

2. **בנאי ההעתקה:** חייבים אותו בגלל פונקציות כמו filter, apply שכן הפונקציות האלו מחזירות רשימה חדשה שמכילה עותקים של האיברים, ולכן חייבת להיות דרך לשכפל את איברי הT לרשימה החדשה, ובאופן כללי יותר ביקשת מאיתנו ליצור עותקים משלנו לכל האובייקטים שנתקבל מהמשתמש ולכן כבר מכך ניתן להניח כי אנחנו חייבים בנאי העתקה.

.2

החזרת T& מאפשרת למשתמש לשנות את הערך של האיבר, מעבר לזה שזה פוגע בעיקרון האינקפסיוליישן זה גם שובר את המיון של הרשימה, כי האיבר משתנה אבל נשאר באותו מקום בשרשרת, והרשימה לא יודעת שהיא צריכה למיין את עצמה מחדש, ולכן היא תישאר לא ממויינת ובכך נהרס כל מבנה הנתונים שלנו והוא הפך לסתם רשימה מקושרת.

.3

מכיוון שהמספר n ידוע רק בזמן ריצה, לא ניתן להשתמש בפונקציה רגילה שמאוחסן בscope שלה המספר n, כלומר להכריז על n בתוך הפוקנציה ולשים בו ערך, ובטח שלא להכריז על n ולשים בו ערך בזמן ריצה, שכן זה אומר שהפונקציה שפילטר תקבל תהיה עם 2 משתני כניסה, ואנחנו צריכים משתנה כניסה 1, אמנם ניתן פשוט להשתמש במשתנה גלובלי ולשים לו ערך בזמן ריצה אבל לא היינו רוצים ליצור משתנה גלובלי n ולתת לו ערך בזמן הריצה, כי זה תכנות רע מאוד.
הפתרון הוא להשתמש בפונקציית LAMBDA ש"תופסת" את המשתנה n מהסביבה הלוקלית שלה באמצעות הסוגריים המרובעים []. זה מאפשר ללמבדה להשתמש בערך הספציפי של n בתוך הפונקציה, כלומר ככה n יהיה בscope שלה.

```
;list.filter([n](T element) { return element % n == 0; })
```

הערה:

יכלנו גם להשתמש בFunction OBJ אבל כבר הצענו רעיון שעובד, וגם קצת מיותר ליצור עכשיו קלאס שלם רק בשביל פונקציה אחת שצריכה רק ערך אחד נוסף, זה סתם עוד שורות קוד מיותרות, אבל יכול להיות שזה היה מעט קריא יותר..